Design and Analysis of Algorithms

CSE Semester 5 — Academic Notes

Synthetic academic knowledge-base document created for CampusAI RAG evaluation and demonstration.

Unit 1 — Algorithm Analysis

- An algorithm is a finite, well-defined sequence of steps that transforms input into output.
- Time complexity describes how running time grows with input size. Space complexity describes additional memory requirements.
- Big-O gives an asymptotic upper bound, Big-Omega gives an asymptotic lower bound, and Big-Theta gives a tight asymptotic bound.
- Common growth rates include O(1), O(log n), O(n), O(n log n), O(n^2), O(2■), and O(n!).
- Best-case, average-case, and worst-case analyses describe behavior under different input conditions.

Unit 2 — Divide and Conquer

- Divide-and-conquer algorithms divide a problem into smaller subproblems, solve them recursively, and combine their results.
- Binary search repeatedly halves a sorted search interval and has O(log n) time complexity.
- Merge sort divides an array into halves, recursively sorts them, and merges the sorted halves. Its time complexity is O(n log n).
- Quick sort partitions an array around a pivot and recursively sorts the partitions. Its average complexity is O(n log n), while a poor pivot strategy can lead to O(n^2) worst-case behavior.
- Recurrences are commonly used to analyze recursive algorithms. The Master Theorem provides solutions for several divide-and-conquer recurrences.

Unit 3 — Greedy Algorithms

- Greedy algorithms build a solution incrementally by selecting a locally optimal choice at each step.
- Activity selection chooses a maximum-size set of mutually compatible activities by repeatedly selecting the activity with the earliest finishing time.
- Fractional knapsack can be solved greedily by selecting items according to value-to-weight ratio.
- Minimum spanning trees can be constructed using Kruskal’s or Prim’s algorithm.
- Dijkstra’s algorithm finds shortest paths from a source in graphs with non-negative edge weights.

Unit 4 — Dynamic Programming

- Dynamic programming solves problems with overlapping subproblems and optimal substructure by storing results of subproblems.
- Two common approaches are memoization (top-down) and tabulation (bottom-up).
- The 0/1 knapsack problem differs from fractional knapsack because each item is either selected completely or not selected.

- Longest Common Subsequence (LCS) uses a recurrence based on whether the final characters of the two sequences match.
- Fibonacci numbers can be computed efficiently with dynamic programming instead of repeatedly recomputing the same recursive subproblems.

Unit 5 — Graphs and Complexity

- Graphs consist of vertices and edges. They may be directed or undirected and weighted or unweighted.
- Breadth-First Search (BFS) explores vertices level by level and can find shortest paths in unweighted graphs. Depth-First Search (DFS) explores as far as possible before backtracking.
- Topological sorting produces an ordering of vertices in a directed acyclic graph (DAG) such that every directed edge goes from an earlier to a later vertex.
- Complexity classes help characterize computational difficulty. P contains problems solvable in polynomial time; NP contains problems whose solutions can be verified in polynomial time.
- An NP-complete problem is both in NP and NP-hard. Reductions are used to relate computational problems.

Quick Revision Checklist

- Know the key definitions and distinguish closely related concepts.
- Practice explaining each concept in your own words before using it in an exam.
- For numerical or algorithmic topics, work through at least one example by hand.
- For database and software-engineering topics, connect definitions to practical examples.

CampusAI RAG metadata suggestion: department=CSE, semester=5, subject=Design and Analysis of Algorithms